

LogiTrack — Documentación Técnica

Base de Datos II · 2026

¿Qué es LogiTrack?

Sistema de gestión logística para una empresa de transporte de encomiendas. Permite registrar envíos, asignarlos a viajes con choferes y vehículos, hacer seguimiento del estado de cada paquete y gestionar incidentes en ruta.

Roles del sistema:

- **Administrador** → acceso total
 - **Empleado** → gestiona envíos y despacho en su sucursal
 - **Chofer** → ve sus viajes e incidentes
 - **Cliente** → rastrea sus envíos y genera nuevos
-

Arquitectura MVC

El sistema separa responsabilidades en tres capas para evitar el "código espagueti":

Capa	Carpeta	Qué hace
Modelo	<code>models/</code>	Accede a la base de datos (SQL, PDO)
Vista	<code>views/</code>	Muestra el HTML al usuario
Controlador	<code>controllers/</code>	Recibe la petición, llama al modelo y pasa datos a la vista

El flujo es: `router.php` → `Controller` → `Model` → `View`

Archivos de entrada:

- `admin/router.php` → panel administrativo
 - `cliente/router.php` → portal del cliente
 - `rastrear.php` → página pública de rastreo (sin login)
-

1. Normalización

Qué es: Organizar las tablas para eliminar redundancia de datos.

Por qué se hizo: Evitar repetir información (ej: el nombre de una sucursal) en múltiples filas. Si cambia, se actualiza en un solo lugar.

Cómo se aplicó:

- El nombre del cliente no se repite en cada envío → existe la tabla `Cliente` y el envío solo guarda el `dni`
- El nombre de la sucursal no está en `Envio` → existe `Sucursal` con su `id_sucursal`

- Los estados del envío no son texto libre → existe `EstadoEnvio` con `id_estado`
- El tipo de incidente está normalizado en `TipoIncidente`
- El tipo de vehículo está en `TipoVehiculo`

Dónde está: En el diseño de las tablas de la base de datos. Las relaciones se ven en los JOINS de los modelos (`models/Envio.php`, `models/Viaje.php`, etc.)

2. Sesiones

Qué es: Mecanismo de PHP para recordar quién está logueado mientras navega el sistema.

Por qué se hizo: El sistema tiene 4 roles con permisos distintos. Sin sesiones, no se podría saber quién está logueado ni qué puede ver.

Cómo se aplicó:

```
// Al hacer login exitoso:
$_SESSION['id_usuario'] = $usuario['id_usuario'];
$_SESSION['id_rol']     = $usuario['id_rol'];
$_SESSION['nombre']     = $usuario['nombre'];

// En cada página protegida:
if (!isset($_SESSION['id_usuario'])) {
    header("Location: /admin/login.php");
    exit();
}

// Control por rol:
if ($_SESSION['id_rol'] !== 1) {
    // acceso denegado
}
```

Dónde está:

- Login: `controllers/AuthController.php`
 - Validación en cada controlador: primeras líneas de cada método `listado()`, `crear()`, etc.
 - Vistas de login: `views/auth/login_admin.php`, `views/auth/login_cliente.php`
-

3. Stored Procedures (Procesos Almacenados)

Qué son: Bloques de lógica SQL guardados en la base de datos que se ejecutan con un solo `CALL`.

Por qué se hicieron: Encapsulan operaciones complejas con múltiples pasos en la base de datos. Además incluyen transacciones internas para garantizar consistencia.

Cómo se aplicaron:

`SP_RegistrarNuevoEnvio` → crea un envío y registra automáticamente el primer estado en `HistorialEstado`:

```
CALL SP_RegistrarNuevoEnvio(nro_tracking, dni_remitente, dni_dest, id_suc_origen, ...);
```

SP_CambiarEstadoEnvio → cambia el estado de un envío e inserta el registro en el historial:

```
CALL SP_CambiarEstadoEnvio(nro_tracking, nuevo_id_estado, id_usuario, observacion);
```

Ambos usan **START TRANSACTION / COMMIT / ROLLBACK** internamente.

Dónde está:

- Definición SQL: `sql/SP_Triggers_Auditoria.sql`
- Llamada desde PHP: `controllers/EnvioController.php` método `crear()` y en el cambio de estado

4. Triggers

Qué son: Acciones automáticas que la base de datos ejecuta cuando ocurre un INSERT, UPDATE o DELETE en una tabla.

Por qué se hicieron: Registrar automáticamente en la tabla `auditoria` cada cambio que ocurre en el sistema, sin que el programador tenga que acordarse de hacerlo manualmente.

Cómo se aplicaron: Hay 14 triggers en la base de datos. Por ejemplo, cuando se modifica un envío:

```
CREATE TRIGGER trg_envio_update
AFTER UPDATE ON envio
FOR EACH ROW
BEGIN
    INSERT INTO auditoria (tabla, id_registro, accion, datos_anteriores,
    datos_nuevos, fecha_hora)
    VALUES ('envio', NEW.nro_tracking, 'UPDATE',
    JSON_OBJECT('id_suc_destino', OLD.id_suc_destino),
    JSON_OBJECT('id_suc_destino', NEW.id_suc_destino),
    NOW());
END;
```

Tablas con triggers: `envio`, `cliente`, `usuario`, `sucursal`, `viaje`, `incidente`, `empleado`

Dónde está:

- Definición SQL: `sql/SP_Triggers_Auditoria.sql`
- Los resultados se ven en: `views/admin/auditoria.php` (panel de auditoría del admin)

5. Vistas (SQL Views)

Qué son: Consultas SQL guardadas en la base de datos con un nombre, que se usan como si fueran tablas.

Por qué se hicieron: Los JOINS entre envío, cliente, sucursal y estado son complejos y se repiten en varios lugares. La vista los encapsula una vez y el PHP hace `SELECT * FROM vista_....`

Cómo se aplicaron:

Vista	Qué une	Usada en
<code>vista_envios_completos</code>	envío + 2x cliente + 2x sucursal + historial + estado	<code>Envio::obtenerTodos()</code> , <code>Envio::obtenerPorSucursal()</code>
<code>vista_viajes_completos</code>	viaje + empleado + sucursal + vehiculo + tipo	<code>Viaje::obtenerTodos()</code>
<code>vista_viajes_activos</code>	igual pero filtra solo los en curso	<code>api</code> / panel
<code>vista_incidentes_abiertos</code>	incidente + tipo + viaje + chofer, solo abiertos	<code>Incidente::obtenerAbiertos()</code>
<code>vista_resumen_por_sucursal</code>	envío agrupado por sucursal y estado	reportes

Antes:

```
// JOIN manual de 5 tablas, 20 líneas de SQL en el modelo
$sql = "SELECT e.nro_tracking, cr.nombre AS nombre_remitente, ...
        FROM Envio e LEFT JOIN Cliente cr ON ... LEFT JOIN Sucursal so ON ...";
```

Después:

```
// Una línea, legible y mantenible
return $this->pdo->query("SELECT * FROM vista_envios_completos ORDER BY
fecha_creacion DESC")->fetchAll();
```

Dónde está:

- Definición SQL: `sql/vistas.sql`
- Uso en PHP: `models/Envio.php`, `models/Viaje.php`, `models/Incidente.php`

6. Seguridad

Qué es: Conjunto de técnicas para proteger el sistema de accesos no autorizados y ataques.

Por qué se hizo: Cualquier sistema web está expuesto a ataques si no se implementan medidas básicas.

Cómo se aplicó:

a) Contraseñas hasheadas:

```
// Al crear usuario:  
$hash = password_hash($password, PASSWORD_BCRYPT);  
  
// Al verificar login:  
password_verify($password_ingresada, $hash_guardado);
```

Las contraseñas nunca se guardan en texto plano en la base de datos.

b) Prepared Statements (prevención de SQL Injection):

```
// MAL (vulnerable):  
$sql = "SELECT * FROM usuario WHERE email = '$email'";  
  
// BIEN (seguro):  
$stmt = $pdo->prepare("SELECT * FROM usuario WHERE email = :email");  
$stmt->execute([':email' => $email]);
```

Todos los inputs del usuario pasan por prepared statements con PDO.

c) XSS prevention:

```
// Toda salida de datos de la DB se escapa:  
echo htmlspecialchars($dato_de_db);
```

d) Control de acceso por rol:

```
// Cada controlador verifica el rol antes de ejecutar:  
if (!in_array($_SESSION['id_rol'], [1, 2])) {  
    header("Location: /admin/login.php"); exit();  
}
```

Dónde está: En todos los controladores (`controllers/`) y en `views/` donde se muestran datos.

7. CRUD

Qué es: Create (crear), Read (leer), Update (actualizar), Delete (eliminar) — las 4 operaciones básicas sobre datos.

Por qué se hizo: Es el requisito fundamental de cualquier sistema de gestión.

Cómo se aplicó en cada entidad:

Entidad	Create	Read	Update	Delete
Envío	crear_envio	gestionar_envios	editar_envio	Anular (soft delete)
Viaje	crear_viaje	ver_viaje / listado	editar_viaje	Cancelar
Incidente	crear_incidente	listado	editar_incidente	Eliminar
Sucursal	form en listado	listado	form en listado	Desactivar / Reactivar
Usuario/Cliente	registro	ver_cliente	editar datos	Desactivar cuenta
Vehículo	form en listado	listado	form en listado	Dar de baja

Dónde está: `controllers/EnvioController.php`, `controllers/ViajeController.php`, `controllers/IncidenteController.php`, `controllers/SucursalController.php`, `controllers/UsuarioController.php`

8. Transacciones

Qué son: Un conjunto de operaciones SQL que se ejecutan todas juntas o ninguna (todo o nada).

Por qué se hicieron: Si al registrar un envío falla la inserción del historial de estado inicial, el envío quedaría en un estado inválido. La transacción garantiza que ambas operaciones ocurran o ninguna.

Cómo se aplicó:

En PHP con PDO:

```
try {
    $this->pdo->beginTransaction();

    // Operación 1: insertar el envío
    $stmt1->execute([...]);

    // Operación 2: registrar estado inicial en historial
    $stmt2->execute([...]);

    $this->pdo->commit(); // todo OK → confirmar
} catch (Exception $e) {
    $this->pdo->rollBack(); // algo falló → deshacer todo
}
```

En Stored Procedures con MySQL:

```
START TRANSACTION;
    INSERT INTO envio (...) VALUES (...);
    INSERT INTO historialestado (...) VALUES (...);
COMMIT;
-- Si hay error: ROLLBACK automático
```

Dónde está:

- PHP: `controllers/EnvioController.php` método `crear()`
- MySQL: `sql/SP_Triggers_Auditoria.sql` dentro de los stored procedures

9. Índices

Qué son: Estructuras que MySQL crea internamente para acelerar las búsquedas en columnas específicas.

Por qué se hicieron: Sin índices, MySQL recorre toda la tabla fila por fila para encontrar un registro. Con índices, lo encuentra directamente como buscando en el índice de un libro.

Cómo se aplicaron:

```
-- Columnas que se filtran frecuentemente:
CREATE INDEX IF NOT EXISTS idx_envio_tracking    ON envio(nro_tracking);
CREATE INDEX IF NOT EXISTS idx_envio_remitente    ON envio(dni_remitente);
CREATE INDEX IF NOT EXISTS idx_historial_tracking ON
historialestado(nro_tracking);
CREATE INDEX IF NOT EXISTS idx_viaje_chofer      ON viaje(legajo_chofer);
CREATE INDEX IF NOT EXISTS idx_incidente_viaje   ON incidente(cod_viaje);
CREATE INDEX IF NOT EXISTS idx_auditoria_tabla   ON auditoria(tabla, fecha_hora);
-- (20 índices en total)
```

Dónde está: `sql/indices.sql`

Estructura de archivos

```
LogiTrack/
├── config/
│   └── db.php          → conexión PDO a MySQL
├── models/
│   ├── Envio.php       → queries de envíos (usa vistas SQL)
│   ├── Viaje.php       → queries de viajes (usa vistas SQL)
│   ├── Incidente.php   → queries de incidentes (usa vistas SQL)
│   ├── Cliente.php     → queries de clientes
│   ├── Paquete.php     → queries de paquetes
│   └── Auditoria.php   → registro de cambios
├── controllers/
│   └── AuthController.php → login, logout, sesiones
```

```

├── EnvioController.php      → CRUD envíos
├── ViajeController.php      → CRUD viajes
├── IncidenteController.php  → CRUD incidentes
├── SucursalController.php   → CRUD sucursales
├── UsuarioController.php    → gestión de usuarios/clientes
├── VehiculoController.php   → gestión de vehículos
├── views/
│   ├── layouts/             → header.php y footer.php compartidos
│   ├── admin/               → vistas del panel admin
│   ├── cliente/             → vistas del portal cliente
│   ├── chofer/              → vistas del panel chofer
│   ├── empleado/           → vistas del panel empleado
│   └── auth/                → login y registro
├── sql/
│   ├── vistas.sql           → CREATE VIEW (5 vistas)
│   ├── indices.sql          → CREATE INDEX (20 índices)
│   └── SP_Triggers_Auditoria.sql → stored procedures y triggers
├── admin/
│   └── router.php           → enrutador del panel admin
├── cliente/
│   └── router.php           → enrutador del portal cliente
├── rastrear.php             → página pública de rastreo
└── index.php                → página de inicio

```

Flujo de una operación típica

Ejemplo: Admin crea un nuevo envío

1. Admin completa el formulario en `views/admin/envios/gestionar.php`
2. El formulario hace POST a `admin/router.php?pagina=crear_envio`
3. `router.php` instancia `EnvioController` y llama a `crear()`
4. El controller valida los datos y llama `CALL SP_RegistrarNuevoEnvio(...)` mediante PDO
5. El stored procedure ejecuta una transacción: INSERT en `envio` + INSERT en `historialestado`
6. Los triggers `trg_envio_insert` y `trg_historial_insert` registran el cambio en `auditoria`
7. El controller redirige al listado con mensaje de éxito
8. El admin puede ver el nuevo envío en `vista_envios_completos` (vista SQL)